Отчёт по лабораторной работе №3

Математические основы защиты информации и информационной безопасности

Сунь Маосин

Содержание

1 Цель работы **5**

2 Реализация алгоритма **6**

2.1 Подготовка данных 6

2.1.1 Код словаря 6

2.2 Функции преобразования 6

2.2.1 Код функций преобразования 7

2.3 Функция шифрования 7

2.3.1 Код функции шифрования 8

2.4 Функция дешифрования 8

2.4.1 Код функции дешифрования 9

2.5 Тестирование на примере из задания 9

2.5.1 Код тестирования 10

2.5.2 Результат выполнения 11

3 Вывод **12**

Список иллюстраций

2.1 Словарь соответствия букв и чисел 6
2.2 Функции преобразования 7
2.3 Функция шифрования 8
2.4 Функция дешифрования 9
2.5 Код тестирования примера 10
2.6 Результат тестирования 11

Список таблиц

1 Цель работы

Изучить принципы шифрования гаммированием и реализовать алгоритм шифрования с использованием конечной гаммы на языке Python. В работе используется русский алфавит с нумерацией букв от 1 до 33.

2 Реализация алгоритма

2.1 Подготовка данных

Для работы с русским текстом был создан словарь соответствия букв и чисел. Каждой букве русского алфавита присвоен порядковый номер от 1 до 33.

2.1.1 Код словаря

```
# Russian alphabet mapping (standard order)
LETTER_TO_NUM = {
        "а": 1,  "б": 2,  "в": 3,  "г": 4,  "д": 5,  "е": 6,  "ё": 7,  "ж": 8,  "з": 9,
        "и": 10, "й": 11, "к": 12, "л": 13, "м": 14, "н": 15, "о": 16, "п": 17,
        "р": 18, "с": 19, "т": 20, "у": 21, "ф": 22, "х": 23, "ц": 24, "ч": 25,
        "ш": 26, "щ": 27, "ъ": 28, "ы": 29, "ь": 30, "э": 31, "ю": 32, "я": 33
}
```

Рис. 2.1: Словарь соответствия букв и чисел

2.2 Функции преобразования

Для удобства работы были созданы две вспомогательные функции: `text_to_numbers` для преобразования текста в список чисел и `numbers_to_text` для обратного преобразования.

2.2.1 Код функций преобразования

```
def text_to_numbers(text):
    """Convert text to list of numbers using the alphabet mapping"""
    text = text.lower()
    numbers = []
    for char in text:
        if char in LETTER_TO_NUM:
            numbers.append(LETTER_TO_NUM[char])
        else:
            # Skip characters not in alphabet (spaces, punctuation)
            continue
    return numbers
```

```
def numbers_to_text(numbers):
    """Convert list of numbers back to text"""
    text = ""
    for num in numbers:
        if num in NUM_TO_LETTER:
            text += NUM_TO_LETTER[num]
    return text
```

Рис. 2.2: Функции преобразования

2.3 Функция шифрования

Функция `gamma_encrypt` выполняет шифрование текста с помощью гаммы. Алгоритм преобразует текст и гамму в числа, затем складывает их с приведением по модулю 33. При сумме больше 33 применяется операция взятия остатка, с особым случаем когда остаток равен 0 (заменяется на 33).

2.3.1 Код функции шифрования

```
def gamma_encrypt(text, gamma):
    """
    Encrypt text using gamma cipher
    text: plaintext (Russian)
    gamma: key phrase (Russian)
    """
    # Convert text and gamma to numbers
    text_numbers = text_to_numbers(text)
    gamma_numbers = text_to_numbers(gamma)

    print(f"Text numbers: {text_numbers}")
    print(f"Gamma numbers: {gamma_numbers}")

    # Encrypt: (text_num + gamma_num) mod 33
    # Note: mod 33 with numbers 1-33 requires special handling
    result_numbers = []
    gamma_len = len(gamma_numbers)

    for i, text_num in enumerate(text_numbers):
        gamma_num = gamma_numbers[i % gamma_len]  # Repeat gamma if needed
        sum_num = text_num + gamma_num

        # Mod 33 for numbers 1-33
        if sum_num > 33:
            sum_num = sum_num % 33
            if sum_num == 0:
                sum_num = 33

        result_numbers.append(sum_num)

    print(f"Result numbers: {result_numbers}")

    # Convert back to text
    encrypted_text = numbers_to_text(result_numbers)
    return encrypted_text
```

Рис. 2.3: Функция шифрования

2.4 Функция дешифрования

Функция `gamma_decrypt` выполняет дешифрование. Из чисел зашифрованного текста вычитаются числа гаммы. Если результат меньше 1, к нему прибавляется 33 для восстановления

исходного значения.

2.4.1 Код функции дешифрования

```
def gamma_decrypt(cipher, gamma):
    """
    Decrypt text using gamma cipher
    cipher: ciphertext (Russian)
    gamma: key phrase (Russian)
    """
    # Convert cipher and gamma to numbers
    cipher_numbers = text_to_numbers(cipher)
    gamma_numbers = text_to_numbers(gamma)

    print(f"Cipher numbers: {cipher_numbers}")
    print(f"Gamma numbers: {gamma_numbers}")

    # Decrypt: (cipher_num - gamma_num) mod 33
    result_numbers = []
    gamma_len = len(gamma_numbers)

    for i, cipher_num in enumerate(cipher_numbers):
        gamma_num = gamma_numbers[i % gamma_len]    # Repeat gamma if needed
        diff_num = cipher_num - gamma_num

        # Mod 33 for numbers 1-33
        if diff_num < 1:
            diff_num = diff_num + 33

        result_numbers.append(diff_num)

    print(f"Result numbers: {result_numbers}")

    # Convert back to text
    decrypted_text = numbers_to_text(result_numbers)
    return decrypted_text
```

Рис. 2.4: Функция дешифрования

2.5 Тестирование на примере из задания

Для проверки корректности работы алгоритма был использован пример из задания: текст “ПРИКАЗ” и гамма “ГАММА”. Программа выводит промежуточные числа и конечный резуль-

тат.

2.5.1 Код тестирования

```
def test_example_from_assignment():
    """Test the example from the assignment"""
    print("=" * 60)
    print("TESTING EXAMPLE FROM ASSIGNMENT")
    print("=" * 60)

    # Example from the assignment
    text = "П Р И К А З"
    gamma = "Г А М М А"

    print(f"Original text: {text}")
    print(f"Gamma: {gamma}")
    print()

    # Encrypt
    encrypted = gamma_encrypt(text, gamma)
    print(f"\nEncrypted: {encrypted}")

    # Decrypt
    decrypted = gamma_decrypt(encrypted, gamma)
    print(f"\nDecrypted: {decrypted}")

    # Expected result from assignment (for reference only)
    expected = "У С Х Ч Б Л"
    print(f"\nExpected (from assignment): {expected}")
    print(f"Note: Different letter numbering gives different result")

    # Verify that encryption and decryption work correctly
    success = text.lower() == decrypted
    print(f"\nEncryption/Decryption successful: {success}")

    print("\n" + "=" * 60)
```

Рис. 2.5: Код тестирования примера

2.5.2 Результат выполнения

```
============================================================
LABORATORY WORK No. 3
Gamma Cipher (Г а м м и р о в а н и е)
============================================================
1. Run example from assignment
2. Test with your own text
3. Encrypt only
4. Decrypt only
5. Exit
------------------------------------------------------------
============================================================
TESTING EXAMPLE FROM ASSIGNMENT
============================================================
Original text: П Р И К А З
Gamma: Г А М М А

Text numbers: [17, 18, 10, 12, 1, 9]
Gamma numbers: [4, 1, 14, 14, 1]
Result numbers: [21, 19, 24, 26, 2, 13]

Encrypted: у с ц ш б л
Cipher numbers: [21, 19, 24, 26, 2, 13]
Gamma numbers: [4, 1, 14, 14, 1]
Result numbers: [17, 18, 10, 12, 1, 9]

Decrypted: п р и к а з

Expected (from assignment): У С Х Ч Б Л
Note: Different letter numbering gives different result

Encryption/Decryption successful: True

============================================================
```

Рис. 2.6: Результат тестирования

3 Вывод

В ходе лабораторной работы был успешно реализован алгоритм шифрования гаммированием с конечной гаммой. Программа корректно выполняет шифрование и дешифрование текста на русском языке. Эксперимент подтвердил, что при наличии правильной гаммы возможно полное восстановление исходного сообщения.